Part 2: Infrastructure and Foundations of Language Models
Lecture 3: Tokenization and Embedding



Lecturer: Milad Vazan
Department of Computer and Data Science
Faculty of Mathematical Sciences, Shahid Beheshti University

© 2026, Shahid Beheshti University

Where We Are in the Course

The Missing Piece

- Previous chapter: neural networks from scratch $\rightarrow$ backpropagation & optimizers
- But a network only works with **numbers**, not words
- Open question: How do we feed "hello" or a full sentence into a network?

From Text to Numbers

This chapter answers one core question:



> *how does a neural network — which only understands numbers — process a word like "hello" or a sentence like "the weather is nice today"?*

By the end, you'll know exactly what math happens to your text before the first neural network layer ever sees it.

NLP vs. Language Model

Two Terms You Must Separate

Term	Definition
NLP (Natural Language Processing)	Broad AI field: understanding, analyzing, generating human language (spell-check, translation, sentiment, QA, chatbots)
Language Model (LM)	A specific tool inside NLP: estimates the probability of a word sequence — "given the words so far, what's the next word?"

NLP is the umbrella. A language model is one very important tool inside it. Don't confuse the two.

The Language Model, Mathematically

A Language Model Is Just a Function

- Given a sequence $w_1, w_2, \ldots, w_{t-1}$, return a probability distribution over the next word w_t:

$$P(w_t \mid w_1, w_2, \ldots, w_{t-1})$$

- Chain Rule of Probability for a full sentence:

$$P(w_1, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \ldots, w_{t-1})$$

> *This looks simple. But computing it requires a neural network. And a neural network only works with numbers. That's the whole challenge of this chapter.*

The Analogy: The Guessing Game

A Guessing Game with a Friend

- Friend says a word → you guess the next → friend reveals the truth → repeat
- The better you are at guessing the next word, the better your "internal language model"
- A real LM plays this game on **billions of sentences** until it's extraordinarily good

> *This simple ability — next-word prediction — at massive scale (billions of parameters, trillions of training words, Transformer architecture) becomes GPT. Emergent abilities arise from optimizing one simple objective.*

The Big Obstacle

The Fundamental Barrier

- The formula $P(w_t \mid w_1, \ldots, w_{t-1})$ needs a neural network
- A neural network **only works with numbers and vectors** — not words or letters
- So the path from raw text $\rightarrow$ network input must pass through critical stages

> *This is the motivation for the entire chapter. Raw text must become numbers — but numbers that preserve meaning.*

The Three-Stage Pipeline

From Raw Text to Numerical Vectors

1. **Tokenization** — split raw text into smaller units (word, subword, character)
2. **Vectorization** — convert each token into a number or numerical vector
3. **Embedding** — ensure these numbers preserve **meaning and similarity** between words

> *These three stages are the backbone of the chapter. We'll go from the simplest possible idea to today's industrial standards.*

Three Ways to Tokenize the Same Text I

Word vs. Subword vs. Character

Level	Input	Example tokens
Word	`unhappiness`	`unhappiness`
Subword	`unhappiness`	`un` | `happi` | `ness`
Character	`unhappiness`	`u` | `n` | `h` | `a` | `p` | `p` | `i` | `n` | `e` | `s` | `s`

- **Word:** compact sequences, but unseen words can be out of vocabulary (OOV).
- **Subword:** balances sequence length and vocabulary coverage; common in modern LLMs.

Three Ways to Tokenize the Same Text II

- **Character:** tiny vocabulary and handles novel spellings, but creates longer sequences.

How We Got Here

Era	Approach	Limitation
1950s–1990s	Hand-written linguistic rules	Fragile — one exception breaks the system
1990s–2013	Statistical methods (e.g., TF-IDF)	No real "meaning," just counts
2013+ (Word2Vec)	Learning-based embeddings	Machine extracts semantic similarity automatically
2017+ (Transformer)	Contextual embeddings	Word meaning is **dynamic**, depends on sentence

A Brief History of NLP II

NLP existed long before deep learning. But the game changed when machines could extract meaning from raw data without human-defined similarity rules.



Why Numbers at All?

- Every neuron does one simple operation:

$$z = \mathbf{w}^T \mathbf{x} + b$$

- Non-negotiable premise: $\mathbf{x}$ must be a vector of real numbers
- Computers have no innate understanding of " " or "book" — only bits and numbers
- Even UTF-8 encoding is useless for meaning: no relation between a letter's code and the word's meaning

> *The Unicode code of " " has nothing to do with the code of " " that would tell us "dog" and "cat" are both pets. Raw character encoding is meaningless for semantics.*

The Naive Idea (and Why It Fails)

"Just Give Each Word a Number!"

- Tempting: apple = 1, banana = 2, airplane = 3
- **Fatal flaw:** numbers 1 and 2 have a natural order and distance
- But there's no reason "apple" should be closer to "banana" than to "airplane"
- A network might wrongly learn that "airplane" (3) lies between "apple" (1) and a word with code 5



Arbitrary numbering creates false numerical relationships. Solving this properly is the rest of the chapter.

Three Requirements of a Good Representation

What a Good Numerical Representation Needs

- **Processability:** matrix multiplication, addition, differentiation must be meaningful
- **Meaning preservation (ideal):** semantically close words (king/queen) must be close in numerical space
- **Computational & memory efficiency:** with vocabularies of hundreds of thousands of words, no memory explosion

> *The history of NLP is the story of trying to satisfy all three. Early methods only satisfied the first.*

Distance in Vector Space

Measuring Similarity

Euclidean Distance

$$d_{\text{Euclidean}}(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_{i=1}^{d} (u_i - v_i)^2}$$

- Direct generalization of the Pythagorean theorem to d dimensions
- Smaller value = closer vectors

Simple and intuitive. But it has a problem for word embeddings.

The Problem with Euclidean Distance

Why Euclidean Distance Fails for Word Embeddings

- Highly sensitive to the **magnitude** of vectors
- In many embedding algorithms, magnitude correlates with word **frequency**, not meaning
- Two synonyms with very different frequencies can have a large Euclidean distance — even if their **direction** (which encodes meaning) is nearly identical

This motivates the shift from distance to angle.

Cosine Similarity — The Industry Standard

Key idea: direction matters more than magnitude.

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\|\|\mathbf{v}\| \cos\theta$$

$$\text{cosine_sim}(\mathbf{u}, \mathbf{v}) = \cos\theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|} = \frac{\sum_{i=1}^{d} u_i v_i}{\sqrt{\sum_{i=1}^{d} u_i^2}\sqrt{\sum_{i=1}^{d} v_i^2}}$$

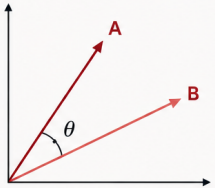
Two arrows on a magnetic board: length doesn't matter, direction does.

Euclidean vs. Cosine

Cosine Similarity

Measures the angle between two vectors (indicating how similar their directions are).

$$\cos(\theta) = \frac{A \cdot B}{||A||\,||B||}$$

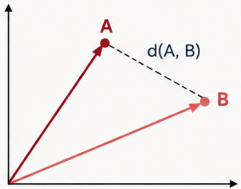


- Values range from -1 to 1.
- 1: same direction (most similar).
- 0: orthogonal (no similarity).
- −1: opposite direction (completely dissimilar).

Euclidean Distance

Measures the actual distance between two vectors (including magnitude).

$$d(A,B) = ||A - B|| = \sqrt{\sum_{i=1}^{n} (a_i - b_i)^2}$$



- Larger distance = less similar.
- Smaller distance = more similar.

Interpreting Cosine Similarity

The Range of Values

Value	Meaning
$+1$	Same direction (perfect similarity)
0	Orthogonal (unrelated direction)
-1	Opposite direction (complete opposition)

A bounded, interpretable range — unlike Euclidean distance, whose range depends on data scale.

Euclidean vs. Cosine: The Beautiful Link

For Normalized Vectors, They're Equivalent

For unit vectors ($\|\mathbf{u}\| = \|\mathbf{v}\| = 1$):

$$\|\mathbf{u} - \mathbf{v}\|^2 = 2(1 - \text{cosine_sim}(\mathbf{u}, \mathbf{v}))$$

Minimizing Euclidean distance = maximizing cosine similarity. That's why vector databases (the basis of RAG) often normalize first, then use fast Euclidean search — conceptually identical to cosine similarity.

Why Cosine Is the Industry Choice

Why Cosine Similarity Wins

- Indifferent to absolute magnitude (which can be confounded by frequency)
- Fast and scalable — especially with pre-normalized vectors
- Bounded, interpretable output range $[-1, 1]$

Classical Vectorization

Classical Vectorization Methods

One-Hot Encoding

- Vocabulary of $|V|$ unique words $\rightarrow$ each word gets a vector of length $|V|$
- All zeros except a single 1 at the word's position:

$$(\mathbf{e}_i)_j = \begin{cases} 1 & j = i \\ 0 & j \neq i \end{cases}$$

Example: $V = \{\text{apple}, \text{banana}, \text{airplane}, \text{book}\}$

$$\text{apple} \rightarrow [1, 0, 0, 0], \quad \text{banana} \rightarrow [0, 1, 0, 0], \ldots$$

Like taking attendance in a class of 50 students — one vector per student.

The Fatal Flaw of One-Hot

Proof That One-Hot Is Semantically Useless

$$\text{cosine_sim}(\mathbf{e}_i, \mathbf{e}_j) = \frac{\mathbf{e}_i \cdot \mathbf{e}_j}{\|\mathbf{e}_i\|\|\mathbf{e}_j\|} = \frac{0}{1 \times 1} = 0 \quad \forall i \neq j$$

- Similarity of "apple" and "banana" (both fruits) = similarity of "apple" and "airplane" = **0**
- All one-hot vectors are equally distant (orthogonal)
- Plus: huge dimensionality, extreme sparsity, no handling of new/unknown words (OOV)

> *This is why one-hot fails. It carries identity, not meaning.*

Bag of Words — A Bag Full of Words, No Order

$$(\mathbf{v}_d)_i = \text{count}(w_i, d)$$

Example:

- Doc 1: "I like apples and I like bananas" $\rightarrow [1, 1, 0, 2, 1, 0, 2, 1, 1]$
- Doc 2: "I don't like airplanes" $\rightarrow [0, 0, 1, 1, 0, 1, 1, 0, 0]$

Imagine writing a sentence, cutting words onto slips, and dropping them in a bag — order is lost, only counts remain.

The Problem with BoW

BoW Ignores Word Order

- "Dog bit cat" and "Cat bit dog" $\rightarrow$ **identical** BoW vectors
- But meanings are completely different (who bit whom?)
- This limitation motivated methods that preserve at least some order

> *This is the fundamental limitation that n-grams try to address.*

n-grams — A Partial Fix for Order

- Instead of counting single words (unigram), count consecutive sequences of n words

For "dog bit cat":

- **Unigram:** {dog, bit, cat}
- **Bigram:** {(dog,bit), (bit,cat)}
- **Trigram:** {(dog,bit,cat)}

> *With bigrams, "dog bit cat" and "cat bit dog" now produce different vectors. Local order is preserved.*

The Curse of Dimensionality

- Vocabulary of $|V|$ words $\rightarrow$ up to $|V|^2$ possible bigrams, $|V|^3$ possible trigrams
- For $|V| = 50{,}000$: **2.5 billion** possible bigrams!
- Most never occur in real text $\rightarrow$ severe sparsity
- Increasing n becomes unmanageable fast $\rightarrow$ rarely use $n > 3$ in practice

This is why n-grams don't scale to capture long-range order.

TF-IDF: The Key Insight

TF-IDF — Smart Weighting of Words

- Stop words ("the", "and", "is") appear frequently in almost every document but carry no discriminative information
- **Core idea:** A word's importance in a document should:
  - **increase** with its frequency in that document
  - **decrease** with its frequency across the whole corpus

TF-IDF Formulas

The Two Components

Term Frequency:

$$\text{TF}(w, d) = \frac{\text{count}(w, d)}{\sum_{w' \in d} \text{count}(w', d)}$$

Inverse Document Frequency:

$$\text{IDF}(w, D) = \log\left(\frac{N}{|\{d \in D : w \in d\}|}\right)$$

Final:

$$\boxed{\text{TF-IDF}(w, d, D) = \text{TF}(w, d) \times \text{IDF}(w, D)}$$

If a word appears in all N documents, $\log(1) = 0$ — its IDF is exactly zero.

Why the Logarithm?

Why Log?

- Word in all N documents $\rightarrow \log(1) = 0 \rightarrow$ zero importance
- Word in 1 out of 1000 documents $\rightarrow \log(1000/1) \approx 6.9$
- Logarithm makes importance grow in a **compressed, controlled** way

Numerical Example ($N = 1000$):

- "apple": TF=0.03, IDF=log(20)≈2.996, TF-IDF≈0.0899
- "the": TF=0.1, IDF=log(1)=0, TF-IDF=0

> *The logarithm prevents very rare words from dominating.*

Smoothed IDF and Implementation

$$\text{IDF}_{\text{smooth}}(w, D) = \log\left(\frac{N+1}{|\{d \in D : w \in d\}| + 1}\right) + 1$$

Smoothing avoids division by zero for unseen words.

TF-IDF Still Has the BoW Limitation

TF-IDF's Fundamental Limitation

- Still built on the BoW vector
- **Ignores word order**
- **No semantic similarity** between different words
- Cosine similarity of "apple" and "banana" = same as "apple" and "airplane" (unless they co-occur)

> *This is exactly the gap that **embedding** fills — and we're now fully equipped to understand it.*

The Concept of Embedding

Mapping Words to Meaningful Vector Space

The Distributional Hypothesis

"You Shall Know a Word by the Company It Keeps"

— John Rupert Firth, 1957

- Words appearing in **similar contexts** have **similar meanings**
- "dog" and "cat" appear in similar sentences: "...loves me", "...eats food", "...is in the house"
- "dog" and "democracy" rarely share contexts
- **Quantifiable:** "context" = words within a small window around the target word

> *This idea — from Firth and Harris — is the mathematical foundation of all modern embedding methods.*

Embedding: Formal Definition

Embedding as a Learnable Mapping

$$E : V \to \mathbb{R}^d, \qquad E(w_i) = \mathbf{v}_i \in \mathbb{R}^d$$

- Usually $d \ll |V|$ (e.g., $d = 300$ vs. $|V| = 100{,}000$)
- Implementation: an **embedding matrix**

$$\mathbf{E} \in \mathbb{R}^{|V| \times d}$$

- Row i = embedding vector of word i
- Lookup = multiplying one-hot vector by the matrix:

$$\mathbf{v}_i = \mathbf{e}_i^T \mathbf{E}$$

> *In practice we never actually do the matrix multiply — it's just row lookup (a fast table lookup).*

A Critical Computational Note

Embedding = A Linear Layer

- $\mathbf{e}_i$ has only one nonzero element $\rightarrow \mathbf{e}_i^T\mathbf{E} =$ just picking row i
- In deep learning libraries: `embedding_matrix[i]` — a simple index operation
- Mathematically equivalent to a **linear layer** with $\mathbf{W} = \mathbf{E}^T$ and no bias
- Useful for understanding how gradients flow (backpropagation from the previous chapter)

> *This equivalence is important: the embedding layer is just another linear layer from a mathematical standpoint.*

The Embedding Matrix Is Learned

The Embedding Matrix Is a Learnable Parameter

- $\mathbf{E}$ is **not** pre-defined or hand-crafted
- Like weights $\mathbf{W}^{[l]}$ from the previous chapter
- Updated via backpropagation and gradient descent
- **Key question:** With what objective do we train it so values become meaningful?

> *This question is exactly what Word2Vec and GloVe answer.*

Why Dimensionality Reduction Doesn't Lose Information

Compression That Improves, Not Loses

- Naive worry: compressing from 100,000-dim one-hot to 300-dim must lose information
- But one-hot **never had semantic information** to lose — only $\log_2(|V|)$ bits of identity
- Embedding uses co-occurrence statistics to build a **dense, meaningful** space
- Analogy: compressing an image by understanding its patterns, not storing every pixel

Degrees of freedom argument: language has massive redundancy (synonyms, inflections). Underlying semantic structure fits in far fewer dimensions.

Linear Compositionality — The "Magic" Equation

$$E(\text{king}) - E(\text{man}) + E(\text{woman}) \approx E(\text{queen})$$

- The difference vector "king − man" encodes the abstract concept of royalty/power
- Adding it to "woman" lands near "queen"
- Embedding space encodes **structural relations** as linear directions

No classical method (one-hot, BoW, TF-IDF) can reproduce this.

Solving Analogies with Cosine Similarity

Analogy as a Search Problem

$$\hat{w} = \arg\max_{w \in V} \text{cosine_sim}\big(E(w),\ E(\text{king}) - E(\text{man}) + E(\text{woman})\big)$$

- Compute the target vector
- Search the whole vocabulary for the closest word by cosine similarity

> *This uses exactly the tool we built earlier in the chapter.*

Honest Caveat

The Magic Isn't Perfect

- Linear compositionality is an **approximately statistical property**, not an exact mathematical law
- Works well for simple analogies: gender, tense, singular/plural, capital/country
- Often fails for complex or abstract relations
- The exact mathematical explanation for why it emerges is still an active research question

> *Be honest about this. It's impressive but not universal.*

Embedding as Matrix Factorization

A Unifying View (Levy & Goldberg, 2014)

- **Almost all** embedding methods can be seen as factorizing a large statistical matrix into two smaller dense matrices:

$$\mathbf{M} \approx \mathbf{U}\mathbf{V}^T$$

- $\mathbf{M}$: a $|V| \times |V|$ co-occurrence/statistical matrix
- Rows of $\mathbf{U}$ (or $\mathbf{V}$) are the final embedding vectors
- Explains why Word2Vec, GloVe, and classical co-occurrence methods produce **similar results**

> *This is one of the deepest theoretical insights in word embeddings.*

A Known Limitation: Anisotropy

- Embedding spaces (static or contextual) are often **anisotropic**
- Word vectors cluster in a **narrow cone** rather than spreading uniformly
- Result: cosine similarity between unrelated words is artificially high (not near zero)
- **Fix:** subtract the mean of the embedding space before computing cosine similarity

Static vs. Contextual Embeddings

A Fundamental Limitation to Know Now

- **Static embeddings** (Word2Vec, GloVe): one fixed vector per word
- Problem: "bank" in "I went to the bank" vs. "I sat on the river bank" $\rightarrow$ same vector!
- Best case: a weighted average of all meanings

> *The solution — contextual embeddings — comes with Transformer and Self-Attention in later chapters.*

Why Static Embeddings Still Matter

- Even in GPT and other modern LMs, the **first layer is exactly the static embedding matrix** $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$
- Contextual embedding takes this initial static vector as a starting point, then enriches it
- **Nothing you learn here becomes useless** — it's layer zero of modern architectures

This is crucial motivation. The static embedding is the foundation of every modern LM.

How to Evaluate an Embedding

- **Intrinsic evaluation:** directly tests vector space properties
  - Word analogy tests (king/queen)
  - Comparing computed similarity with human-rated similarity (WordSim-353)
- **Extrinsic evaluation:** applies embedding to a real downstream task
  - Text classification, NER, sentiment analysis
  - Often more reliable — what matters is practical usefulness

Extrinsic is usually the more trustworthy measure.

Word2Vec

Learning Embeddings from Raw Text

CBOW — Continuous Bag of Words

Intuition: Fill in the blank — "I went to ____ today and bought a book"

Architecture (very simple — one linear hidden layer):

1. Map each context word via input embedding matrix $\mathbf{W}_{in}$
2. Average the context vectors:

$$\mathbf{h} = \frac{1}{2c} \sum_{-c \le j \le c, j \neq 0} E(w_{t+j})$$

3. Multiply by output matrix: $\mathbf{z} = \mathbf{W}_{out}^{T} \mathbf{h}$
4. Softmax over vocabulary:

$$P(w_t \mid \text{context}) = \frac{\exp(z_{w_t})}{\sum_{j=1}^{|V|} \exp(z_j)}$$

Same Softmax + Cross-Entropy we proved in the previous chapter.

Skip-gram — The Reverse of CBOW

Intuition: Given "bookstore", what words likely appear nearby? (book, bought, went, shelf)

Objective: For each center word w_t, maximize the average log-probability of all context words:

$$J(\theta) = \frac{1}{T}\sum_{t=1}^{T}\sum_{-c\le j\le c, j\neq 0} \log P(w_{t+j} \mid w_t)$$

$$P(w_{t+j} \mid w_t) = \frac{\exp(\mathbf{v}'^{T}_{w_{t+j}}\mathbf{v}_{w_t})}{\sum_{k=1}^{|V|}\exp(\mathbf{v}'^{T}_{k}\mathbf{v}_{w_t})}$$

$\mathbf{v}_{w_t}$ *is the input embedding of the center word;* $\mathbf{v}'_{w_{t+j}}$ *is the output embedding of the context word.*

CBOW vs. Skip-gram



The Big Computational Problem

The Softmax Denominator Problem

- Denominator requires summing over **all** $|V|$ vocabulary words
- For a vocabulary of 1 million words $\rightarrow$ 1 million exponentials per gradient update!
- Completely infeasible for training on massive text (billions of words)

> *This is the computational bottleneck that Negative Sampling solves.*

The Genius Solution — Negative Sampling

- Replace "a $|V|$-class classification" with a **simple binary classification**:
  - "Did this (center, context) pair really co-occur in text, or not?"

$$J_{\text{NEG}} = \log \sigma(\mathbf{v}'^{T}_{w_O} \mathbf{v}_{w_I}) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n(w)} \left[\log \sigma(-\mathbf{v}'^{T}_{w_i} \mathbf{v}_{w_I}) \right]$$

- k: number of negative samples (usually 5–20)
- $P_n(w) \propto \text{freq}(w)^{3/4}$

For $k = 5$, $|V| = 1{,}000{,}000$: about 200,000× faster than full Softmax.

Why This Connects to Previous Chapter

Consistency with the $\hat{y} - y$ Pattern

- Negative Sampling converts the heavy $|V|$-class problem into $k+1$ light binary problems
- Complexity drops from $O(|V|)$ to $O(k)$
- The gradient pattern matches the $\hat{y} - y$ form from the previous chapter's Cross-Entropy proof

This isn't an arbitrary trick — it's a principled approximation.

GloVe

Global Statistics + Vector Learning

GloVe: The Math

The GloVe Objective

- Co-occurrence matrix $\mathbf{X}$: X_{ij} = number of times word j appears near word i
- Key insight: **ratios** of co-occurrence probabilities carry more meaning than raw probabilities
- Simplified to:

$$\mathbf{v}_i^T \tilde{\mathbf{v}}_k + b_i + \tilde{b}_k = \log(X_{ik})$$

- Weighted least-squares loss:

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \left(\mathbf{v}_i^T \tilde{\mathbf{v}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

- Weighting: $f(x) = (x/x_{\max})^\alpha$ for $x < x_{\max}$, else 1 (usually $x_{\max} = 100$, $\alpha = 3/4$)

> *The weighting prevents very frequent pairs from dominating the loss.*

Comparison

Feature	Word2Vec	GloVe
Information source	Local text windows	Global co-occurrence matrix
Learning type	Prediction-based	Regression-based (count-based)
Quality on rare words	Usually weaker	Better

Despite different angles, both solve a similar mathematical problem — matrix factorization.

A Note for Today

Relevance in the Modern Era

- With **contextual embeddings**, direct industrial use of Word2Vec/GloVe has decreased
- But deep understanding remains essential — the input embedding matrix is the **first layer of every modern LLM** (including GPT)

> *Foundational concepts never go out of style.*

Modern Tokenization

Modern Tokenization Methods

BPE — Byte Pair Encoding

- Originally a **data compression** algorithm from the 1990s
- Reborn in 2016 for tokenization

Algorithm:

1. Split training text into individual characters
2. Count occurrences of every adjacent character/subword pair
3. Merge the most frequent pair into a new unit
4. Replace all occurrences with the new unit
5. Repeat until reaching the target vocabulary size

> *The algorithm learns meaningful linguistic units (like the suffix "-est") without any human linguistic rules.*

BPE: Numerical Example

BPE in Action

Corpus: `low` (5×), `lower` (2×), `newest` (6×), `widest` (3×)

- Most frequent adjacent pair: `e`+`s` (6+3 = 9) → merge to `es`
- Next: `es`+`t` (9) → merge to `est`
- Next: `l`+`o` (5+2 = 7) → merge to `lo`
- ...

Result: The algorithm automatically discovers meaningful units like `est` (superlative suffix) — **without any human linguistic programming!**

> *This is the remarkable emergent behavior of BPE.*

WordPiece

- Instead of the most frequent raw pair, choose the pair whose merge gives the greatest increase in **training data likelihood**:

$$\text{score}(x, y) = \frac{\text{freq}(xy)}{\text{freq}(x) \times \text{freq}(y)}$$

> *This formula resembles **Pointwise Mutual Information**: "How much more than random chance do these two units co-occur?"*

SentencePiece

- Completely removes dependency on pre-tokenization/whitespace
- Treats raw text as a Unicode sequence
- Supports BPE and also **Unigram Language Model**:

$$P(x) = \prod_{i=1}^{M} P(x_i)$$

- Estimated via EM algorithm; picks the most probable subword segmentation

> *Particularly useful for languages without clear whitespace boundaries.*

Special Tokens I

What They Are and Why We Need Them

Token	Purpose	Example / Why?
`[CLS]`	Provides a representation of the input for classification in models like BERT.	Classify a sentence as positive or negative.
`[SEP]`	Marks the boundary between sentences or text segments.	Separates a question from a sentence.
`[PAD]`	Fills shorter sequences so inputs in a batch have equal length.	Adds padding to shorter sentences.
`[UNK]`	Represents a token the tokenizer cannot recognize or encode.	Handles an unrecognized word.
`[MASK]`	Hides a token that the model must predict during masked language modeling.	`I love [MASK]` $\rightarrow$ `NLP`

Special Tokens II

Key Idea

Special tokens provide structural or task-specific information beyond the ordinary text.

Special tokens have specific roles in model inputs and training. Not every model uses all these tokens; their use depends on the tokenizer and model architecture.

Golden Rules of Tokenization

Four Golden Rules

1. Always use a tokenizer from the same family as the model
2. Pay attention to the maximum token length
3. Use Padding and Attention Mask correctly
4. Be aware of language differences — an English tokenizer performs poorly on Persian

Practical advice that saves hours of debugging.

Chapter Summary

Concept	Role
Euclidean / Cosine similarity	Base tools for measuring similarity in vector space
One-hot / BoW / n-gram / TF-IDF	Classical, sparse, no semantic similarity
Embedding	Maps words to dense, meaningful vector space
Word2Vec / GloVe	Learning embeddings from prediction or global statistics
BPE / WordPiece / SentencePiece	Subword tokenization to solve OOV

The Road Ahead

What's Next

- The natural next step: **Self-Attention and the Transformer architecture**
- Where these static embedding vectors become **dynamic contextual vectors**

> *Everything we learned here is the foundation for what comes next.*

Mathematical Appendix

For the Rigorous Reader

Appendix: Skip-gram Gradient

Full Gradient of Skip-gram with Full Softmax

$$\mathcal{L} = -\log P(w_O \mid w_I) = -\mathbf{v}'^{T}_{w_O}\mathbf{v}_{w_I} + \log \sum_{k=1}^{|V|} \exp(\mathbf{v}'^{T}_{k}\mathbf{v}_{w_I})$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{v}_{w_I}} = -\mathbf{v}'_{w_O} + \sum_{k=1}^{|V|} P(k \mid w_I)\mathbf{v}'_k$$

$$\boxed{\frac{\partial \mathcal{L}}{\partial \mathbf{v}_{w_I}} = \sum_{k=1}^{|V|} \Big(P(k \mid w_I) - \mathbb{1}[k = w_O]\Big)\mathbf{v}'_k}$$

Exactly the $\hat{y} - y$ pattern — but this time as a weighted linear combination of all output vectors. This is the source of the $O(|V|)$ cost.

Appendix: Negative Sampling ≈ NCE

Negative Sampling as Noise Contrastive Estimation

- NCE goal: turn density estimation into binary classification
- "Did this sample come from real data P_d or noise P_n?"

$$P(D = 1 \mid w, c) = \frac{P_d(w \mid c)}{P_d(w \mid c) + kP_n(w)}$$

- Negative Sampling approximates $P_d(w \mid c)$ with $\exp(\mathbf{v}'^T_w \mathbf{v}_c)$ (without full Softmax normalization)
- Yields:

$$P(D = 1 \mid w, c) \approx \sigma(\mathbf{v}'^T_w \mathbf{v}_c)$$

As $k \to \infty$, the NS gradient converges to the exact Softmax gradient — a controllable approximation, not an arbitrary trick.

Appendix: Word2Vec = PMI Matrix Factorization

Equivalence of Skip-gram and PMI Matrix Factorization

- Levy & Goldberg (2014): At the global optimum, Skip-gram with Negative Sampling is equivalent to factorizing:

$$\mathbf{v}_w'^T \mathbf{v}_c = \text{PMI}(w, c) - \log k$$

- Where:

$$\text{PMI}(w, c) = \log \frac{P(w, c)}{P(w)P(c)}$$

Proof sketch: For a fixed pair (w, c), set the derivative of the local NS objective w.r.t. $x = \mathbf{v}_w'^T \mathbf{v}_c$ to zero:

$$x^* = \log \frac{\#(w, c) \cdot |D|}{\#(w) \cdot \#(c)} - \log k = \text{PMI}(w, c) - \log k$$

This proves Word2Vec is, mathematically, a shifted PMI matrix factorization — the unifying view from earlier.

Appendix: GloVe Derivation from Symmetry

Deriving the GloVe F Function from First Principles

Initial assumption: $F(\mathbf{v}_i, \mathbf{v}_j, \tilde{\mathbf{v}}_k) = P_{ik}/P_{jk}$

Constraint 1: F must depend only on vector differences (for $i \leftrightarrow j$ symmetry):

$$F(\mathbf{v}_i - \mathbf{v}_j, \tilde{\mathbf{v}}_k) = \frac{P_{ik}}{P_{jk}}$$

Constraint 2: Arguments must become scalar (via dot product):

$$F\big((\mathbf{v}_i - \mathbf{v}_j)^T \tilde{\mathbf{v}}_k\big) = \frac{P_{ik}}{P_{jk}}$$

Constraint 3: Symmetry between word and context ($i \leftrightarrow k$) forces F to be a homomorphism $F(a - b) = F(a)/F(b) \rightarrow$ only solution (given continuity) is the exponential:

$$F = \exp \quad \Longrightarrow \quad \mathbf{v}_i^T \tilde{\mathbf{v}}_k + b_i + \tilde{b}_k = \log X_{ik}$$

Appendix: Cauchy-Schwarz Bound

Why Cosine Similarity Is in $[-1, 1]$

Cauchy-Schwarz inequality:

$$|\mathbf{u} \cdot \mathbf{v}| \leq \|\mathbf{u}\|\|\mathbf{v}\|$$

Divide both sides by $\|\mathbf{u}\|\|\mathbf{v}\|$:

$$\left| \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|} \right| \leq 1 \quad \Longrightarrow \quad -1 \leq \text{cosine_sim}(\mathbf{u}, \mathbf{v}) \leq 1$$

- Equality $= 1$ iff $\mathbf{u} = \lambda \mathbf{v}$, $\lambda > 0$ (same direction)
- Equality $= -1$ iff $\lambda < 0$ (opposite direction)

The $[-1, 1]$ range is not a convention — it's a direct consequence of a linear algebra theorem.

BPE: Entropy Analysis and Huffman Connection

- BPE was historically designed for data compression
- Shannon's information theory: optimal code length for a symbol with probability p is $-\log_2 p$ bits
- Merging the most frequent pair (highest p) greedily moves toward reducing total code length
- This is the same principle Huffman coding solves **optimally** (not greedily)

Key difference:

- Huffman: optimal coding tree for **fixed** symbols
- BPE: expands the symbol set at each step (adds merged symbols)
- BPE therefore does **both** "learning meaningful linguistic units" and "statistical compression"; Huffman only does the latter

> *This connects tokenization to information theory — a deep and beautiful link.*

Thank You

Questions?

- *Every neural network needs numbers — text must be converted first.*
- *Classical methods (one-hot, BoW, TF-IDF) are simple but semantically empty.*
- *Embeddings (Word2Vec, GloVe) learn meaning from data.*
- *Modern tokenization (BPE, WordPiece, SentencePiece) solves OOV.*
- *All of this is layer zero of every modern LLM.*